

GnarOne Game Math

Preset catalogue and customizability guide for the Rollerz SDK integration

2026-09-29 · build df1a0ac · <https://danvid1009.github.io/gnarone-game-math>

1. How presets work

Every engine is published as a browser module that implements the RGS provider contract and rolls locally. The SDK creates a game from a **preset name**, never from raw parameters:

```
import { createGame } from 'https://danvid1009.github.io/gnarone-game-math/<game>/api/rgs.js';
const g = createGame({ preset: 'MEDIUM' });
const s = g.open(); // session, balance, chip levels, preset tables
let r = g.bet({ sessionId: s.sessionId, betAmount: 1000, betType: 'MEDIUM' });
r = g.nextAction({ roundId: r.roundId, actionCode: 'CONTINUE' }); // multi-step games only
g.collect({ roundId: r.roundId });
```

A preset is a frozen configuration: the game parameters, the target RTP and the chip levels. Each preset is its own folder under **<game>/api/<preset>/** holding **field.json** (the config, the derived tables and the settlement rule), **rounds.json** and **rounds/NNN.json** (100 pre-drawn rounds from seeds fixture-000 to fixture-099, settled for every bet type) and a worked **sample-request / sample-response** pair. **index.json** at the game level lists presets, parameters and the realised RTP over the fixtures. Passing **seed** on a bet reproduces any fixture round through the live calls.

Money is always an integer in minor units. The bet response carries the balance minus the stake; collect adds the win. Every engine has exact RTP by construction for every bet type and, for the multi-step games, for every stopping rule; the Node test suites verify this by summation and by simulation.

Three levels of customization

level	what changes	who does it	turnaround
Runtime	bet type and bet amount per round; which preset the client opens	client / SDK	none
Preset	a new row in a parameter table below (payout array, Elo field, top multiplier, RTP...)	GnarOne, via the preset config	same day: config, RTP verification, fixtures, publish
Structural	new mechanics or rules outside the parameter tables (payout structure, deck design, step rules)	GnarOne, ticket	design + math + tests

Anything not in a parameter table is structural. Anything in a table is a preset request: we add the row, verify the RTP exactly, generate the fixtures and publish it under its own folder.

Global parameters (every engine)

parameter	type	default	constraint
rtp	number	per engine	$0 < \text{rtp} < 1$; other constraints listed per engine
levels	integer[] minor units	100, 200, 500, 1000, 2500, 5000, 10000	non-empty; a bet must equal one of them
startingBalance	integer	100000	demo wallet in the sandbox only

2. Stepper

A fair crash ladder (RTP 0.97). Ten rungs; the player CONTINUEs or CASH_OUTs after each; one uniform drawn at bet time fixes where the ladder breaks. Survival to rung k is RTP / c_k , so the first step carries the whole house edge and every later step is a fair bet: every stopping rule returns exactly the RTP. Multi-step wire shape (CONTINUE, CASH_OUT, COLLECT).

Parameters

parameter	type	default	constraint
rtp	number	0.97	
maxWin	number	(preset)	greater than 1; sets $c_k = \text{round}(\text{maxWin}^{k/\text{steps}}, 2)$
steps	integer	10	1 to 20 sensible
returns	number[]	derived	alternative to maxWin/steps: strictly increasing, c_1 greater than rtp

Presets

preset	top win	rungs	rung 1 pays	P(instant crash)	P(reach the top)
EASY	10x	10	1.26x	23%	9.7%
MEDIUM	25x	10	1.38x	29.7%	3.9%
HARD	100x	10	1.58x	38.6%	1%

Who each preset is for

EASY	first-time players; the ladder is survivable and the top is reachable one round in ten
MEDIUM	the default; meaningful climbs with a 25x ceiling
HARD	high-variance players; every rung is a real risk, 100x once in a hundred rounds

Customizability

Runtime: the bet type is the preset name (the live Stepper sends difficulty as the bet type), so a client that offers three difficulties opens three games. Bet amount from the level list.

Preset: any top multiplier, any step count, any RTP. The rung schedule is derived, or an explicit list of rungs can be given instead (for example a slow start and a steep finish). The only constraint is that the first rung pays more than the RTP, otherwise the instant-crash probability would be negative.

Structural (ticket): anything that changes the survival rule, for example a non-zero payout on a crash, insurance, or a bonus rung. Those change the strategy-proofness argument and need a new derivation.

3. Single Shot

One draw, one payout (RTP 0.95). The unit interval is cut into bands whose widths are the probabilities; inside a band the paid multiple ramps linearly from $a - r$ to $a + r$ with $r = 0.1 \times$ the smallest gap between positive payouts, so the mean of each band is exactly its listed payout and the RTP is exact by summation. A zero band pays exactly 0. Single-call wire shape.

Parameters

parameter	type	default	constraint
rtp	number	0.95	strictly between the smallest and largest payout
payouts	number[]	(preset)	distinct, non-negative, at most one 0, at least two positive
method	geometric maxent weights	geometric	how probabilities are assigned to hit the RTP
weights	number[]	-	method weights only: relative frequencies of the winning payouts (the entry for 0 is ignored); the probability of the 0 payout is then computed so the RTP is exact
fraction	number	0.1	band half-width as a fraction of the smallest gap; fixed, not exposed

Presets

preset	payouts (x stake)	P(win)	stdev	band probabilities
classic	0, 0.5, 1, 2, 5, 20	51.1%	2.82x	0x 48.9% / 0.5x 25.5% / 1x 13.2% / 2x 6.9% / 5x 3.6% / 20x 1.9%
steady	0, 0.5, 1, 1.5, 2, 3	71.9%	0.9x	0x 28.1% / 0.5x 22% / 1x 17.3% / 1.5x 13.6% / 2x 10.6% / 3x 8.3%
spiky	0, 1, 2, 5, 20, 100	33.5%	5.65x	0x 66.5% / 1x 22.3% / 2x 7.5% / 5x 2.5% / 20x 0.8% / 100x 0.3%
always-pays	0.2, 0.5, 1, 2, 5	100%	1.25x	0.2x 39.5% / 0.5x 25.7% / 1x 16.8% / 2x 10.9% / 5x 7.1%

Who each preset is for

classic	the reference table; wins half the time, 20x top
steady	low variance; wins 72% of spins, nothing above 3x
spiky	high variance; wins a third of the time, rare 100x
always-pays	never returns zero; the top is capped at 5x

Customizability

Runtime: one bet type (BASE); only the amount varies.

Preset: the payout array itself is the input, so a new table is a preset row, not a design change: send the multiples you want and the RTP, we publish the folder. Geometric is the default probability method (each higher payout is a fixed factor rarer); maxent gives the flattest distribution that hits the RTP; weights lets you dictate the relative frequency of the winning payouts, and the engine then sets the probability of the 0 payout so the RTP is exact ($p_0 = 1 - \text{RTP} / E[\text{payout} \mid \text{win}]$). Any RTP between the smallest and largest payout works.

Direct input: because the RTP is exact by construction for any valid array, the SDK may also pass **payouts** (and optionally **method** and **weights**) straight to createGame instead of a preset name; the engine validates the array and throws on an invalid one. Presets exist for the fixtures and for reproducibility, not because other arrays are unsafe.

Structural (ticket): non-linear ramps inside a band, bands that overlap, or a payout that depends on the bet size.

4. Top 3 and Top 1

An Elo race (RTP 0.95). Strength is $10^{\text{Elo}/400}$; one uniform is recycled into the full finishing order (Plackett-Luce), so win, place and show probabilities are closed-form. Top 3 pays stake plus profit for first, a quarter of that profit for second and a fifth for third, with the profit solved per racer so every racer returns exactly the RTP. Top 1 pays the winner only at RTP / P(win). Single-call wire shape; the bet type is the racer name.

Parameters

parameter	type	default	constraint
rtp	number	0.95	
racers	{ name, elo }[]	(preset)	3 to 20 racers
mode	top3 top1	top3	Top 1 is the same field paid winner-only
structure	{ place, show }	1/4, 1/5	Top 3 only: fractions of the win profit paid for 2nd and 3rd

Presets

preset	racers	Elo range	favourite wins	outsider wins	Top 3 multipliers, favourite	Top 1 odds fav / outsider
field-12	12	1850 to 1300	27%	1.1%	1.8x / 1.2x / 1.16x	3.52x / 83.53x
field-8	8	1800 to 1380	31.2%	2.8%	1.48x / 1.12x / 1.1x	3.05x / 34.19x
field-6	6	1750 to 1400	36.4%	4.9%	1.24x / 1.06x / 1.05x	2.61x / 19.57x
field-12-tight	12	1650 to 1452	13.8%	4.4%	3.86x / 1.71x / 1.57x	6.87x / 21.47x

Who each preset is for

field-12	the reference field: clear favourite, long shots at the back
field-8	fewer choices with a similar odds spread
field-6	short field for quick rounds; shortest prices
field-12-tight	near-even field; Top 1 odds cluster between 7x and 21x, Top 3 pays 3.9x for the favourite

Customizability

Runtime: the bet type is any racer name from the preset; the client can show the odds from open() before the first bet.

Preset: any number of racers with any Elo values, so a new field is a preset row. For Top 3 we check one feasibility condition: the favourite's probability of finishing in the first three must stay below the RTP (field-12 sits at 66.4%, field-6 at 83.5%); a field with a very dominant favourite fails it and we tell you the largest Elo gap that works. The place and show fractions (1/4, 1/5) can be changed per preset. Top 1 has no feasibility condition.

Structural (ticket): exotic bets (exacta, trifecta), fixed-odds tables that do not return the RTP per racer, in-race cash-out, or racer-specific edges.

5. Rip and Rumble

Card packs from two decks (workbook Math-rippin-rumble-ver4). A pack is five cards drawn without replacement; each rarity pays count x value x multiplier(count), an exact hypergeometric sum. Each bet type draws from the low deck with the one probability that makes the expected win divided by the cost equal the RTP. Bundles give eleven packs for the price of ten. Single-call wire shape.

Parameters

parameter	type	default	constraint
rtp	number	0.95	must lie between the two decks' own returns; the low/high mix is solved
credit	integer minor units	100	price of one credit; a BASE pack costs 5 credits
betTypes	table	BASE 5 / BOOSTED 10 / BASE_BUNDLE 5x10 (11 packs) / BOOSTED_BUNDLE 10x10 (11)	cost in credits, packs dealt, packs paid
decks, pays, multipliers	workbook tables	ver4	structural

Presets

preset	RTP	P(low deck) BASE	BOOSTED	BASE_BUNDLE	BOOSTED_BUNDLE	stdev (x stake) BASE / BOOSTED
sheet-v4	0.95	87.3%	8.2%	94.5%	22.6%	1.62 / 1.49
sheet-v4-0.93	0.93	89%	11.6%	96%	25.6%	1.58 / 1.47
sheet-v4-0.97	0.97	85.7%	4.9%	93%	19.6%	1.67 / 1.51

Who each preset is for

sheet-v4	the reference sheet at 0.95
sheet-v4-0.93	tighter margin; same feel
sheet-v4-0.97	looser margin; same feel

Customizability

Runtime: four bet types; the amount must be a multiple of the pack price (cost x paid packs x credit), so BASE bets are 500, 1000, 2500, 5000 minor units and BASE_BUNDLE 5000, 10000, 25000, 50000.

Preset: the RTP (we re-solve the deck mix, which is why 0.93 and 0.97 are presets), the credit price, and the bet-type table (pack costs, bundle sizes). Note that BOOSTED_BUNDLE is not in the current live SDK bet-type list; drop it from the preset or add it to the SDK.

Structural (ticket): deck composition, per-rarity pays, the count multipliers, pack size, or a third deck. Each changes the hypergeometric tables and needs re-verification against a new workbook.

6. Game of Three

Rock n Smash (workbook Rock n Smash.xlsx). Three rocks; a fixed table gives how many break (0 to 3) and pays 5 / 1 / 0.5 / 0; on two or three breaks a gem fires with a fixed chance and multiplies the win by 2x to 10x from a weighted table (mean 3.667). The RTP is the exact sum over 16 outcome zones. Single-call wire shape.

Parameters

parameter	type	default	constraint
rtp	number	as sheet	if set, the win table is scaled so both modes return it
modes	rows (count probabilities, gem chance, wins)	BASE, BOOSTED from the workbook	structural
gems	weighted table 2x to 10x	workbook	structural

Presets

preset	BASE RTP	BASE hit rate	BASE stdev	BOOSTED RTP	BOOSTED hit rate	BOOSTED stdev
sheet	97.07%	76%	1.84x	97.15%	88%	1.6x
rtp-0.95	95%	76%	1.8x	95%	88%	1.56x

Who each preset is for

sheet	exact workbook numbers, 97.07% and 97.15%
rtp-0.95	the same game at a 95% return

Customizability

Runtime: two bet types, BASE and BOOSTED; BOOSTED is the SDK's boosted bet type.

Preset: the RTP target. We keep the workbook's hit probabilities and gem chances and scale the win amounts, so the feel of the game is unchanged and only the payouts move.

Structural (ticket): different break probabilities, gem odds or the gem multiplier table, more rocks, or a separate bonus round.

7. Pick One

An n-way bet with exact RTP (0.95). Each option owns a zone of the unit interval of width p ; backing option j pays RTP / p_j on a hit, so every option returns exactly the RTP regardless of how the probabilities are chosen. Single-call wire shape; the bet type is the option name.

Parameters

parameter	type	default	constraint
rtp	number	0.95	
n	integer	(preset)	at least 2 equal options
weights	number[]	-	alternative to n; positive, any scale
labels	string[]	HEADS / TAILS or OPTION_i	one per option

Presets

preset	options (probability, odds)
coin	HEADS (50%, 1.9x), TAILS (50%, 1.9x)
three	OPTION_1 (33.3%, 2.85x), OPTION_2 (33.3%, 2.85x), OPTION_3 (33.3%, 2.85x)
five	OPTION_1 (20%, 4.75x), OPTION_2 (20%, 4.75x), OPTION_3 (20%, 4.75x), OPTION_4 (20%, 4.75x), OPTION_5 (20%, 4.75x)
weighted	RED (50%, 1.9x), GREEN (30%, 3.17x), BLUE (20%, 4.75x)

Who each preset is for

coin	two-way 1.9x flip
three	three equal doors at 2.85x
five	five equal doors at 4.75x
weighted	unequal odds on labelled options, 1.9x to 4.75x

Customizability

Runtime: the bet type is any option name; the amount from the level list.

Preset: any number of options, equal or weighted, any labels, any RTP. This is the most freely customizable engine: a preset is literally the option list.

Direct input: as with Single Shot, the SDK may pass **n** or **weights** and **labels** directly to createGame; the odds are exact for any input.

Structural (ticket): multi-pick bets, partial payouts for near misses, or options whose probability depends on earlier rounds.

8. Storage Auction (provisional)

The Stepper ladder with a locker on top (RTP 0.95). CONTINUE is a bid, an outbid is the crash, CASH_OUT is the hammer: the player takes the locker at its current value and opens it, and the contents are a Single Shot draw whose mean is exactly 1, so every take rule still returns the RTP. The locker draw is the recycled residual of the same uniform. Wire shape identical to Stepper plus a **locker** field on the final state. The numbers are provisional while the design is being finalised; the wire shape is final.

Parameters

parameter	type	default	constraint
rtp	number	0.95	
maxWin, steps	as Stepper	steps 10	first bid value greater than rtp
box.payouts	number[]	0.25, 0.5, 1, 2, 5	locker multiples; probabilities solved so the mean is exactly 1
box.method	maxent geometric	maxent	

Presets

preset	top value	bid 1 value	P(outbid on bid 1)	P(reach the top)	largest single win
SMALL	5x	1.17x	18.8%	19%	25.1x (top value x 5x locker)
MEDIUM	10x	1.26x	24.6%	9.5%	50.3x (top value x 5x locker)
LARGE	25x	1.38x	31.2%	3.8%	125.6x (top value x 5x locker)
XL	100x	1.58x	39.9%	0.9%	502.5x (top value x 5x locker)

Who each preset is for

SMALL	gentle ladder, 5x top
MEDIUM	10x top
LARGE	25x top
XL	100x top; a 5x locker at the top pays 500x

Customizability

Runtime: the bet type is the locker size (the preset name); amount from the level list.

Preset: top value, bid count and RTP as in Stepper, plus the locker table: any list of multiples, including 0 for an empty locker, with probabilities solved to mean 1. A flat locker ($X = 1$) reproduces the Stepper exactly.

Structural (ticket): refunds on an outbid, escalating bid sizes with money returned between rounds, or a locker whose mean is not 1. These are under discussion and would be a new engine version rather than a preset.

Appendix A: race fields

Every racer in every Top 3 / Top 1 preset, with the closed-form win probability and the multipliers each mode pays (Top 3: first / second / third; Top 1: winner).

field-12 (12 racers, Elo 1850 to 1300)

racer	Elo	P(win)	Top 1 pays	Top 3 pays 1st / 2nd / 3rd
Ace	1850	27%	3.52x	1.8x / 1.2x / 1.16x
Bolt	1780	18%	5.27x	2.7x / 1.42x / 1.34x
Cinder	1720	12.8%	7.44x	3.95x / 1.74x / 1.59x
Dash	1680	10.1%	9.37x	5.11x / 2.03x / 1.82x
Ember	1640	8.1%	11.8x	6.6x / 2.4x / 2.12x
Flint	1600	6.4%	14.85x	8.5x / 2.87x / 2.5x
Gale	1560	5.1%	18.7x	10.9x / 3.48x / 2.98x
Hex	1520	4%	23.54x	13.94x / 4.24x / 3.59x
Iris	1480	3.2%	29.64x	17.78x / 5.19x / 4.36x
Jolt	1430	2.4%	39.52x	24.01x / 6.75x / 5.6x
Kite	1380	1.8%	52.71x	32.32x / 8.83x / 7.26x
Lux	1300	1.1%	83.53x	51.78x / 13.69x / 11.16x

field-8 (8 racers, Elo 1800 to 1380)

racer	Elo	P(win)	Top 1 pays	Top 3 pays 1st / 2nd / 3rd
Ace	1800	31.2%	3.05x	1.48x / 1.12x / 1.1x
Bolt	1740	22.1%	4.3x	2.02x / 1.25x / 1.2x
Cinder	1680	15.6%	6.08x	2.92x / 1.48x / 1.38x
Dash	1620	11.1%	8.59x	4.34x / 1.84x / 1.67x
Ember	1560	7.8%	12.13x	6.43x / 2.36x / 2.09x
Flint	1500	5.5%	17.14x	9.42x / 3.11x / 2.68x
Gale	1440	3.9%	24.21x	13.69x / 4.17x / 3.54x
Hex	1380	2.8%	34.19x	19.74x / 5.69x / 4.75x

field-6 (6 racers, Elo 1750 to 1400)

racer	Elo	P(win)	Top 1 pays	Top 3 pays 1st / 2nd / 3rd
Ace	1750	36.4%	2.61x	1.24x / 1.06x / 1.05x
Bolt	1680	24.3%	3.9x	1.68x / 1.17x / 1.14x
Cinder	1610	16.3%	5.84x	2.55x / 1.39x / 1.31x
Dash	1540	10.9%	8.74x	4.09x / 1.77x / 1.62x
Ember	1470	7.3%	13.08x	6.53x / 2.38x / 2.11x
Flint	1400	4.9%	19.57x	10.25x / 3.31x / 2.85x

field-12-tight (12 racers, Elo 1650 to 1452)

racer	Elo	P(win)	Top 1 pays	Top 3 pays 1st / 2nd / 3rd
Ace	1650	13.8%	6.87x	3.86x / 1.71x / 1.57x
Bolt	1632	12.5%	7.62x	4.32x / 1.83x / 1.66x
Cinder	1614	11.2%	8.45x	4.85x / 1.96x / 1.77x
Dash	1596	10.1%	9.37x	5.43x / 2.11x / 1.89x
Ember	1578	9.1%	10.4x	6.08x / 2.27x / 2.02x
Flint	1560	8.2%	11.53x	6.81x / 2.45x / 2.16x

racer	Elo	P(win)	Top 1 pays	Top 3 pays 1st / 2nd / 3rd
Gale	1542	7.4%	12.79x	7.63x / 2.66x / 2.33x
Hex	1524	6.7%	14.19x	8.53x / 2.88x / 2.51x
Iris	1506	6%	15.73x	9.53x / 3.13x / 2.71x
Jolt	1488	5.4%	17.45x	10.65x / 3.41x / 2.93x
Kite	1470	4.9%	19.36x	11.89x / 3.72x / 3.18x
Lux	1452	4.4%	21.47x	13.27x / 4.07x / 3.45x

Appendix B: files per game

<https://danvid1009.github.io/gnarone-game-math/<game>/api/rgs.js> – the module (createGame)
<https://danvid1009.github.io/gnarone-game-math/<game>/api/index.json> – presets, parameters, bet types, realised RTP
<https://danvid1009.github.io/gnarone-game-math/<game>/api/<preset>/field.json> – frozen config, derived tables, how_to_settle
<https://danvid1009.github.io/gnarone-game-math/<game>/api/<preset>/rounds.json>, [rounds/NNN.json](https://danvid1009.github.io/gnarone-game-math/<game>/api/<preset>/rounds/NNN.json) – 100 seeded rounds settled for every bet type
<https://danvid1009.github.io/gnarone-game-math/<game>/api/<preset>/sample-request.json>, [sample-response.json](https://danvid1009.github.io/gnarone-game-math/<game>/api/<preset>/sample-response.json) – round 007 through the real calls

Games: stepper, single-shot, top-3, top-1, rip-rumble, game-of-three, pick-one, auction. Each page also carries the full algorithm write-up, figures, and a playable demo.